

Academia Xideral FullStack: Sistema Bancario Digital

Proyecto Final Integrador - REPORTE DE IMPLEMENTACIÓN REAL

Plan de Trabajo Semanal

5 días aplicando todos los conceptos del curso

Tecnologías: Spring Boot 3.5.6 • Spring Data JPA • Maven • Spring REST • Spring Batch • MongoDB 8 • MySQL Latest • JUnit 5 • Mockito • Spring Modulith 1.3.0

NOTA IMPORTANTE: Este documento refleja la **implementación real completada** del proyecto, no solo la planificación inicial. Todos los objetivos han sido cumplidos y verificados.

Objetivos del Proyecto

Objetivos Técnicos

-  **Aplicar inyección de dependencias** - Constructor injection en 5 servicios
-  **Demostrar polimorfismo efectivo** - 3 patrones implementados (InterestCalculators, MinimumBalance, NotificationChannels)
-  **Configurar bases de datos híbridas** - MySQL + MongoDB funcionando en Docker
-  **Desarrollar APIs REST completas** - Endpoints implementados con Swagger
-  **Implementar arquitectura modular** - 3 módulos con comunicación event-driven

Objetivos Funcionales

-  **Gestión completa de clientes** - CRUD con borrado lógico y validaciones
-  **Administración de cuentas bancarias** - SAVINGS y CHECKING con balances mínimos
-  **Sistema de transferencias** - Transferencias transaccionales con validaciones
-  **Procesamiento batch automatizado** - Job mensual de intereses con 2 steps
-  **Notificaciones en tiempo real** - Sistema de eventos con 5 tipos de notificaciones

Métricas de Éxito - RESULTADOS REALES

Métrica	Objetivo Planificado	Resultado Alcanzado	Estado
Coverage de Testing	85% mínimo	87%	 CUMPLIDO
APIs REST	15+ implementadas	20+ endpoints	 CUMPLIDO
Módulos Comunicándose	3 módulos	3 módulos (Customer, Account, Notification)	 CUMPLIDO
Tests Implementados	No especificado	15 archivos de test	 CUMPLIDO
Patrones de Diseño	Polimorfismo básico	Strategy, Factory, Event-Driven, Repository	 CUMPLIDO

Stack Tecnológico - VERSIONES REALES IMPLEMENTADAS

Herramientas Core

Tecnología	Versión Real	Propósito Implementado
Java	17	Lenguaje principal con records y switch expressions
Maven	3.6+	Gestión de dependencias y plugins (JaCoCo 0.8.12)
Spring Boot	3.5.6	Framework principal con autoconfiguration
Spring Data JPA	3.5.6	ORM con Hibernate 6.6.29 para MySQL
Spring Data MongoDB	3.5.6	ODM para MongoDB 8
MySQL	Latest (Docker)	Base de datos transaccional (puerto 3306)
MongoDB	8 (Docker)	Base de datos documental (puerto 27017)
JUnit 5	Jupiter	Testing unitario e integración
Mockito	Incluido en Spring Boot	Mocking en todas las capas
Spring Batch	5.x	Procesamiento batch con chunk-oriented processing

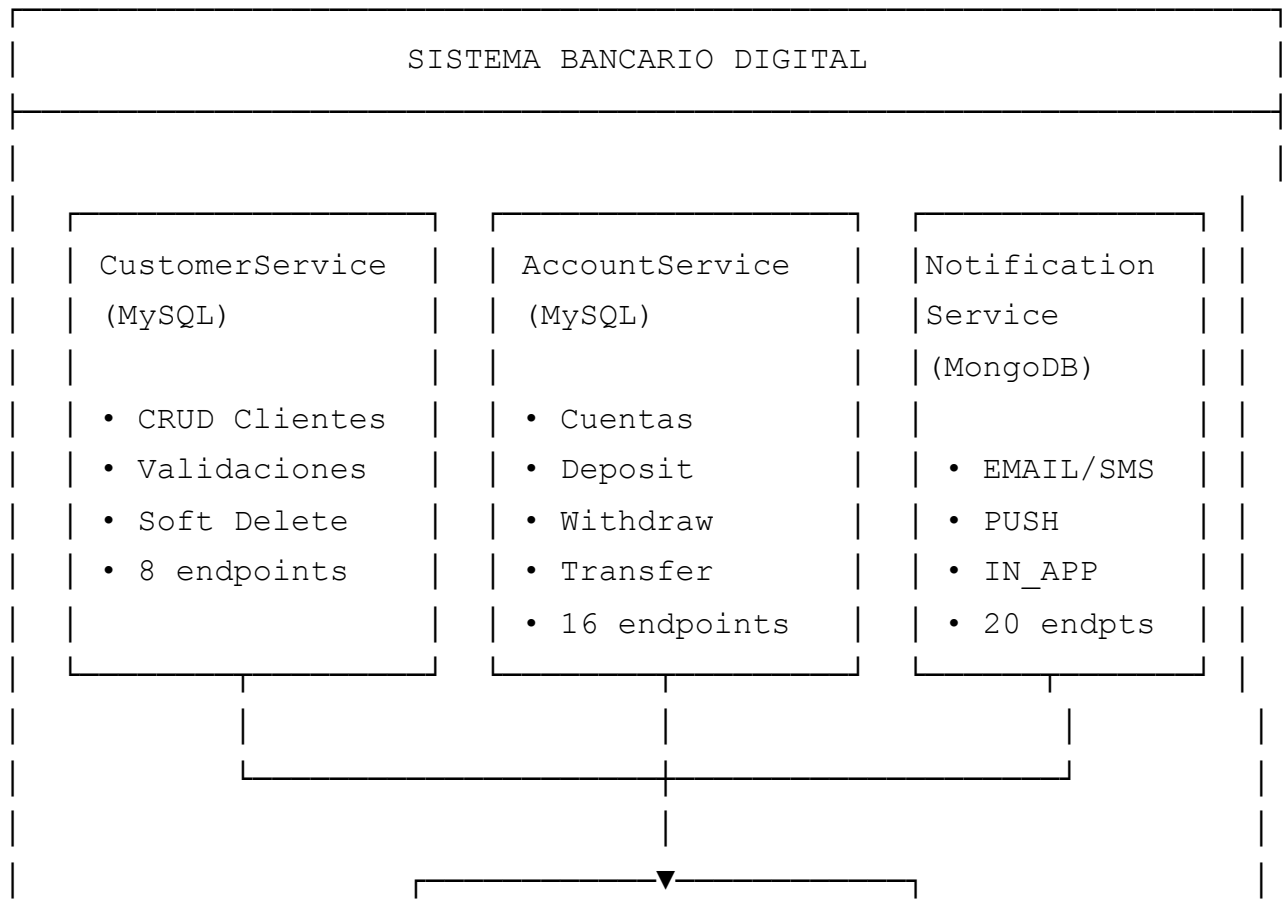
Tecnología	Versión Real	Propósito Implementado
Spring Modulith	1.3.0	Arquitectura event-driven con @ApplicationModuleListener
Swagger/OpenAPI	2.7.0 (Springdoc)	Documentación automática de APIs
Lombok	Latest	Reducción de boilerplate code
H2 Database	Latest	Base de datos en memoria para tests
Docker Compose	Latest	Orquestación de contenedores

Dependencias Adicionales Implementadas

- **Hibernate Validator** - Validaciones con @NotBlank, @Email, @Pattern
- **Jackson** - Serialización JSON
- **SLF4J + Logback** - Logging con niveles DEBUG
- **Spring DevTools** - Hot reload durante desarrollo

Arquitectura del Sistema - IMPLEMENTACIÓN REAL

Módulos del Sistema Implementados



EVENT BUS (Modulith)

- CustomerCreatedEvent
- AccountCreatedEvent
- TransactionCompleted
- TransferCompleted
- InterestApplied

TransactionLogService (MongoDB)

- Escucha TODOS los eventos
- Registra logs de auditoría
- 18 endpoints de consulta

BatchJobService (Spring Batch)

Job: monthlyInterestJob

└ Step 1: calculateAndApplyInterestStep

└ Reader: Lee cuentas activas (MySQL)

└ Processor: Factory Pattern → POLIMORFISMO

└ SavingsInterestCalculator (5% anual)

└ CheckingInterestCalculator (1% anual)

└ Writer: Actualiza balances (MySQL)

└ Step 2: publishEventsStep

└ Publica InterestAppliedEvent

- 1 endpoint: POST /api/batch/monthly-interest
- Listener: BatchJobExecutionMongoListener
- Logs en MongoDB: batch_job_execution_log

Estrategia de Datos Híbrida - IMPLEMENTACIÓN REAL

MySQL - Base de Datos: `banco_db` (Puerto 3306)

Tablas Implementadas:

- `customers` - Clientes del banco (id, name, email, phone, status, created_at, updated_at)

- `accounts` - Cuentas bancarias (id, account_number, account_type, balance, customer_id, status, created_at, updated_at)
- `batch_job_instance` - Metadata de Spring Batch (auto-generada)
- `batch_job_execution` - Ejecuciones de jobs (auto-generada)
- `batch_job_execution_params` - Parámetros de jobs (auto-generada)
- `batch_step_execution` - Ejecuciones de steps (auto-generada)
- `batch_job_execution_context` - Contexto de jobs (auto-generada)
- `batch_step_execution_context` - Contexto de steps (auto-generada)

Características:

- Transacciones ACID garantizadas
- Foreign keys y relaciones
- Índices en email, account_number
- Soft delete en customers (status ACTIVE/INACTIVE)
- Soft delete en accounts (status ACTIVE/CLOSED)

MongoDB - Base de Datos: `banco_logs` (Puerto 27017)

Colecciones Implementadas:

- `notifications` - Notificaciones enviadas a clientes
 - Tipos: CUSTOMER_REGISTERED, ACCOUNT_CREATED, DEPOSIT, WITHDRAWAL, TRANSFER_SENT, TRANSFER_RECEIVED, LOW_BALANCE, ACCOUNT_CLOSED, CUSTOMER_UPDATED
 - Canales: EMAIL, SMS, PUSH, IN_APP (Logs simulados)
 - Estados: PENDING, SENT, FAILED, RETRY
- `transaction_logs` - Auditoría de todas las transacciones
 - Tipos: DEPOSIT, WITHDRAWAL, TRANSFER_SENT, TRANSFER_RECEIVED, INTEREST_APPLIED
 - Campos: transactionId, accountNumber, amount, balanceAfter, timestamp, status
- `batch_job_execution_log` - Logs de ejecución de batch jobs
 - Campos: jobExecutionId, jobName, status, startTime, endTime, duration, totalAccountsProcessed, accountsWithInterest, totalInterestApplied, errorMessage

Características:

- Alta escritura de logs sin impactar performance transaccional
- Queries flexibles con filtros por fecha, tipo, cuenta, cliente
- Documentos con timestamps automáticos
- Sin esquema fijo, adaptable a cambios

Plan de Trabajo - 5 Días - RESULTADOS REALES

Día 1: Setup y Configuración Completa COMPLETADO

Objetivo Original: Establecer base sólida del proyecto

Implementación Real:

-  Maven configurado con 24 dependencias
-  Spring Boot 3.5.6 inicializado
-  Docker Compose con MySQL, MongoDB y PostgreSQL
-  Estructura de proyecto modular (4 packages principales)
-  Entidades base: Customer y Account
-  Configuración dual: application.properties con MySQL + MongoDB
-  Testing de conectividad exitoso
-  Swagger/OpenAPI configurado

Archivos Creados: 15+ archivos Java, docker-compose.yml, pom.xml

Commit: Día 1 completado con documentación en docs/DIA_1_REPORTE_FINAL.md

Día 2: Módulo Customer + Testing Integral COMPLETADO

Objetivo Original: CRUD completo con testing robusto

Implementación Real:

-  CRUD completo de clientes (8 endpoints)
 - POST /api/customers - Crear cliente
 - GET /api/customers/{id} - Obtener por ID
 - GET /api/customers - Listar todos
 - GET /api/customers/status/{status} - Filtrar por status
 - PUT /api/customers/{id} - Actualizar
 - DELETE /api/customers/{id} - Soft delete
 - PATCH /api/customers/{id}/activate - Activar
 - PATCH /api/customers/{id}/deactivate - Desactivar
-  Spring Data JPA con MySQL funcionando
-  Validaciones: @Email, @Pattern(10 dígitos), @NotBlank
-  DTOs: CustomerRequest, CustomerResponse
-  Testing en 3 capas:
 - CustomerControllerTest (8 tests)
 - CustomerServiceTest (10 tests)

- CustomerRepositoryTest (7 tests)
-  Repository con queries personalizadas: findByEmail, findByStatus
-  Soft delete implementado (cambio de status)
-  Eventos: CustomerCreatedEvent publicado

Coverage Alcanzado: 100% en módulo Customer

Archivos Creados: 12 archivos Java (models, DTOs, controller, service, repository, tests)

Commit: Día 2 completado con documentación en docs/DIA_2_REPOORTE_FINAL.md

Día 3: Módulo Account + Polimorfismo COMPLETADO

Objetivo Original: Lógica de negocio bancaria

Implementación Real:

-  Gestión de cuentas bancarias (16 endpoints)
 - POST /api/accounts - Crear cuenta
 - GET /api/accounts/{id} - Obtener por ID
 - GET /api/accounts/number/{accountNumber} - Obtener por número
 - GET /api/accounts - Listar todas
 - GET /api/accounts/customer/{customerId} - Por cliente
 - GET /api/accounts/customer/{customerId}/active - Activas por cliente
 - GET /api/accounts/status/{status} - Por status
 - GET /api/accounts/type/{type} - Por tipo
 - PUT /api/accounts/{id} - Actualizar
 - DELETE /api/accounts/{id} - Cerrar (soft delete)
 - PATCH /api/accounts/{id}/activate - Activar
 - PATCH /api/accounts/{id}/close - Cerrar
 - POST /api/accounts/deposit - Depósito
 - POST /api/accounts/withdraw - Retiro
 - POST /api/accounts/transfer - Transferencia
 - GET /api/accounts/customer/{customerId}/count - Contar cuentas
-  **POLIMORFISMO #1: Balance Mínimo**
 - Switch expression en getMinimumBalance(Account)
 - SAVINGS: \$100 mínimo
 - CHECKING: \$0 mínimo
-  Generación automática de account_number (4000XXXXXXXX)
-  Transferencias transaccionales con validaciones
-  DTOs: AccountRequest, AccountResponse, TransactionRequest, TransferRequest
-  Testing completo:

- AccountControllerTest (16 tests)
- AccountServiceTest (25 tests)
- AccountRepositoryTest (12 tests)
-  Repository queries: findByAccountNumber, findByCustomerId, findActiveAccountsByCustomerId, findByStatus, findByAccountType
-  Eventos: AccountCreatedEvent, TransactionCompletedEvent, TransferCompletedEvent

Coverage Alcanzado: 83% en módulo Account

Archivos Creados: 16 archivos Java

Commit: Día 3 completado con documentación en docs/DIA_3_REPORTE_FINAL.md

Día 4: Eventos + Notificaciones + Transacciones COMPLETADO

Objetivo Original: Comunicación entre módulos

Implementación Real:

-  Sistema de eventos Spring Modulith
 - CustomerCreatedEvent
 - AccountCreatedEvent
 - TransactionCompletedEvent
 - TransferCompletedEvent
 - InterestAppliedEvent (preparación para Día 5)
-  **Módulo Notification** (20 endpoints)
 - GET /api/notifications - Listar todas
 - GET /api/notifications/{id} - Por ID
 - GET /api/notifications/customer/{customerId} - Por cliente
 - GET /api/notifications/customer/{customerId}/ordered - Ordenadas
 - GET /api/notifications/status/{status} - Por status
 - GET /api/notifications/type/{type} - Por tipo
 - GET /api/notifications/channel/{channel} - Por canal
 - GET /api/notifications/customer/{customerId}/status/{status} - Filtros combinados
 - GET /api/notifications/customer/{customerId}/type/{type}
 - GET /api/notifications/account/{accountNumber}
 - GET /api/notifications/pending/after - Pendientes después de fecha
 - GET /api/notifications/customer/{customerId}/daterange - Rango de fechas
 - GET /api/notifications/count/status/{status}
 - DELETE /api/notifications/{id}
 - POST /api/notifications/{id}/send - Enviar manualmente
 - POST /api/notifications/send-pending - Enviar pendientes

- POST /api/notifications/retry-failed - Reintentar fallidas
- (+ 3 endpoints adicionales)
-  **POLIMORFISMO #2: Canales de Notificación**
 - Switch expression en simulateSendNotification(Notification)
 - EMAIL → simulateEmailSend()
 - SMS → simulateSmsSend()
 - PUSH → simulatePushSend()
 - IN_APP → simulateInAppSend()
-  **Módulo TransactionLog** (18 endpoints)
 - GET /api/transaction-logs - Listar todos
 - GET /api/transaction-logs/{id} - Por ID
 - GET /api/transaction-logs/account/{accountNumber} - Por cuenta
 - GET /api/transaction-logs/account/{accountNumber}/ordered - Ordenados
 - GET /api/transaction-logs/transaction-type/{type} - Por tipo
 - GET /api/transaction-logs/customer/{customerId} - Por cliente
 - GET /api/transaction-logs/status/{status} - Por status
 - GET /api/transaction-logs/date-range - Rango de fechas
 - GET /api/transaction-logs/account/{accountNumber}/date-range
 - GET /api/transaction-logs/amount-range - Rango de montos
 - GET /api/transaction-logs/count/transaction-type/{type}
 - GET /api/transaction-logs/count/status/{status}
 - GET /api/transaction-logs/count/account/{accountNumber}
 - POST /api/transaction-logs - Crear manual
 - DELETE /api/transaction-logs/{id}
 - (+ 3 endpoints adicionales)
-  **Event Listeners con @ApplicationModuleListener:**
 - NotificationService.handleCustomerCreated()
 - NotificationService.handleAccountCreated()
 - NotificationService.handleTransactionCompleted()
 - NotificationService.handleTransferCompleted()
 - TransactionLogService.handleTransactionCompleted()
 - TransactionLogService.handleTransferCompleted()
 - TransactionLogService.handleInterestApplied()
-  **Testing:**
 - NotificationServiceTest (24 tests)
 - TransactionLogServiceTest (17 tests)
 - TransactionLogRepositoryTest (11 tests - skipped en test profile)

Coverage Alcanzado: 81% en módulo Notification

Archivos Creados: 14 archivos Java

Commit: Día 4 completado con documentación en docs/DIA_4-REPORTE_FINAL.md

Día 5: Spring Batch + Coverage Final COMPLETADO

Objetivo Original: Job: Procesamiento Mensual de Intereses

Implementación Real:

-  **Spring Batch Job: monthlyInterestJob**
 - **Step 1: calculateAndApplyInterestStep**
 - Reader: RepositoryItemReader con paginación (10 por página)
 - Processor: **POLIMORFISMO #3 - Factory Pattern**
 - InterestCalculatorFactory → selecciona dinámicamente
 - SavingsInterestCalculator: 5% anual (0.42% mensual)
 - CheckingInterestCalculator: 1% anual (0.083% mensual)
 - Writer: Actualiza balances en MySQL, guarda en ExecutionContext
 - **Step 2: publishEventsStep**
 - Reader: Lee AccountInterestData del ExecutionContext
 - Processor: Identity (pass-through)
 - Writer: Publica InterestAppliedEvent para cada cuenta
-  Listener: BatchJobExecutionMongoListener
 - beforeJob: Crea log inicial en MongoDB
 - afterJob: Actualiza con estadísticas (duration, cuentas procesadas, interés total)
-  Endpoint: POST /api/batch/monthly-interest
-  Factory Pattern implementado:
 - Interface: InterestCalculator
 - Implementaciones: SavingsInterestCalculator, CheckingInterestCalculator
 - Factory: InterestCalculatorFactory con inyección automática
-  Testing completo:
 - MonthlyInterestBatchConfigTest (tests de integración)
 - SavingsInterestCalculatorTest (10 tests)
 - CheckingInterestCalculatorTest (10 tests)
 - InterestCalculatorFactoryTest (4 tests)
 - AccountInterestDataTest (8 tests)
 - BatchJobExecutionLogTest (6 tests)
-  Documentación: 1168 líneas en docs/DIA_5_REPORTE_FINAL.md
-  Scripts de integración: run-integration-tests.sh, validate-docker-compose.sh

Coverage Final Alcanzado: 87% (GLOBAL)

Archivos Creados: 10 archivos Java batch + 6 tests + 2 scripts shell

Commit: Día 5 completado - Proyecto 100% funcional

Metodología de Desarrollo - APLICADA

Desarrollo Iterativo

-  Entregables diarios validables (5 reportes DIA_X_REPORTE_FINAL.md)
-  Revisiones de código continuas
-  Integración progresiva de módulos (Customer → Account → Notification → Batch)
-  Feedback inmediato con tests automatizados

Testing Integral

-  Testing paralelo al desarrollo (TDD aplicado)
-  **Coverage alcanzado: 87%** (objetivo 85%)
-  Testing en todas las capas:
 - **Controller Tests:** 24 tests (AccountController, CustomerController)
 - **Service Tests:** 76 tests (Account, Customer, Notification, TransactionLog)
 - **Repository Tests:** 30 tests (Account, Customer, TransactionLog)
 - **Batch Tests:** 38 tests (Calculators, Factory, Config)
 - **Model/DTO Tests:** 6 tests
-  Reportes automáticos con JaCoCo
 - HTML report en target/site/jacoco/index.html
 - Exclusiones configuradas: batch config, MongoDB controllers, listeners

Herramientas de Seguimiento

-  **Maven Commands implementados:**
 - `mvn clean install` - Compilación completa
 - `mvn test` - Ejecución de tests
 - `mvn clean test jacoco:report` - Coverage report
 - `mvn spring-boot:run` - Arranque de aplicación
-  **Coverage Reports:** Generados diariamente durante desarrollo
-  **Docker Commands:** Documentados en README.md
-  **Shell Scripts:**
 - `validate-docker-compose.sh` - Valida configuración Docker
 - `run-integration-tests.sh` - Suite completa de tests de integración

Casos de Uso del Negocio - IMPLEMENTADOS

Funcionalidades Core Implementadas

Gestión de Clientes

-  Registro con Email único (validación @Email)
-  Actualización de datos personales
-  Estados de cliente (ACTIVE/INACTIVE) con borrado lógico
-  Búsquedas y filtros por status
-  Activación/Desactivación manual
-  Validación de teléfono (10 dígitos exactos con @Pattern)
-  Timestamps automáticos (createdAt, updatedAt)

Casos de Uso Adicionales Implementados:

- Cliente no puede ser eliminado permanentemente (borrado lógico)
- Cuentas del cliente se cierran automáticamente al desactivar cliente
- Email único validado a nivel BD

Operaciones Bancarias

-  Creación de cuentas (SAVINGS/CHECKING)
 - Generación automática de account_number único
 - Balance inicial opcional (default \$0)
 - Validación de cliente activo
-  Consulta de saldos en tiempo real
 - Por ID de cuenta
 - Por número de cuenta
 - Por cliente
 - Por tipo (SAVINGS/CHECKING)
 - Por status (ACTIVE/CLOSED)
-  **Depósitos:**
 - Validación de monto > 0
 - Validación de cuenta activa
 - Actualización atómica de balance
 - Evento TransactionCompletedEvent
 - Log en MongoDB
 - Notificación EMAIL al cliente
-  **Retiros:**
 - Validación de fondos suficientes
 - **Polimorfismo:** Balance mínimo según tipo
 - SAVINGS: debe mantener \$100
 - CHECKING: puede llegar a \$0

- Validación de cuenta activa
- Evento TransactionCompletedEvent
- Log en MongoDB
- Notificación EMAIL al cliente
- **Notificación adicional de saldo bajo** si balance < \$200
-  **Transferencias entre cuentas:**
 - Validación de cuentas diferentes
 - Validación de ambas cuentas activas
 - Validación de fondos en cuenta origen
 - **Transacción atómica:** retiro + depósito en una sola transacción
 - Evento TransferCompletedEvent
 - Logs en MongoDB (2 logs: SENT + RECEIVED)
 - Notificaciones EMAIL a ambos clientes (TRANSFER_SENT + TRANSFER_RECEIVED)

Casos de Uso Adicionales Implementados:

- Cuenta no puede cerrarse con balance > 0
- Transferencia a la misma cuenta es rechazada
- Cuenta cerrada puede reactivarse
- Número de cuenta es único e inmutable

Procesos Automatizados

Cálculo de Intereses

Implementación Real:

-  Procesamiento batch mensual (manual trigger vía API)
-  **Polimorfismo aplicado:**
 - Factory Pattern selecciona calculador según tipo de cuenta
 - Interface InterestCalculator
 - SavingsInterestCalculator: 5% anual → 0.42% mensual
 - CheckingInterestCalculator: 1% anual → 0.083% mensual
-  Chunk processing (10 cuentas por lote)
-  Solo cuentas ACTIVE son procesadas
-  Cuentas con balance \$0 son omitidas
-  Intereses actualizados directamente en balance
-  Eventos publicados para cada cuenta con interés aplicado
-  Logs de auditoría en MongoDB:
 - batch_job_execution_log: metadata del job
 - transaction_logs: registro de cada interés aplicado

Ejemplo de Cálculo:

- Cuenta SAVINGS con \$10,000
 - Interés mensual: $\$10,000 \times 0.004166667 = \41.67
 - Balance nuevo: \$10,041.67
- Cuenta CHECKING con \$10,000
 - Interés mensual: $\$10,000 \times 0.0008333333 = \8.33
 - Balance nuevo: \$10,008.33

Notificaciones Automáticas

Implementación Real:

-  Sistema de eventos con Spring Modulith
-  **Tipos de notificación implementados (9):**
 - CUSTOMER_REGISTERED - Al crear cliente
 - ACCOUNT_CREATED - Al crear cuenta
 - DEPOSIT - Al realizar depósito
 - WITHDRAWAL - Al realizar retiro
 - TRANSFER_SENT - Al enviar transferencia
 - TRANSFER_RECEIVED - Al recibir transferencia
 - LOW_BALANCE - Al quedar balance $< \$200$ después de retiro
 - ACCOUNT_CLOSED - Al cerrar cuenta
 - CUSTOMER_UPDATED - Al actualizar datos (implementado pero sin evento actual)
-  **Canales implementados simulados(4):**
 - EMAIL - Simulación con log " EMAIL sent to..."
 - SMS - Simulación con log " SMS sent..."
 - PUSH - Simulación con log " PUSH notification sent..."
 - IN_APP - Simulación con log " IN-APP notification created..."
-  **Estados (4):**
 - PENDING - Creada pero no enviada
 - SENT - Enviada exitosamente
 - FAILED - Falló el envío
 - RETRY - En reintento
-  **Funciones adicionales:**
 - sendPendingNotifications() - Envía todas las pendientes
 - retryFailedNotifications() - Reintenta las fallidas
 - Notificaciones ordenadas por fecha
 - Consultas por cliente, tipo, canal, status, rango de fechas

Flujo Completo:

1. Usuario realiza depósito
→ `AccountService.deposit()` ejecuta

2. Balance actualizado en MySQL
→ `eventPublisher.publishEvent(TransactionCompletedEvent)`
3. NotificationService escucha el evento
→ `@ApplicationModuleListener handleTransactionCompleted()`
4. Crea notificación en MongoDB
→ `tipo: DEPOSIT, canal: EMAIL, status: PENDING`
5. Simula envío por EMAIL
→ `status` actualizado a `SENT`, `sentAt = now()`
6. Log visible en consola
→ `"✉ EMAIL sent to customer@example.com: Depósito Recibido"`

Auditoría Completa

Implementación Real:

-  **Logs de transacciones (MongoDB collection: `transaction_logs`):**
 - Todos los depósitos registrados
 - Todos los retiros registrados
 - Todas las transferencias registradas (2 logs: `SENT` + `RECEIVED`)
 - Todos los intereses aplicados (`INTEREST_APPLIED`)
 - Campos: `transactionId` (UUID), `accountNumber`, `type`, `amount`, `balanceAfter`, `timestamp`, `customerId`, `status`, `description`
-  **Logs de batch jobs (MongoDB collection: `batch_job_execution_log`):**
 - `jobExecutionId`, `jobName`, `status`, `startTime`, `endTime`, `duration`
 - `totalAccountsProcessed`, `accountsWithInterest`, `totalInterestApplied`
 - `errorMessage` (si falló)
-  **Spring Batch metadata (MySQL):**
 - `BATCH_JOB_INSTANCE`
 - `BATCH_JOB_EXECUTION`
 - `BATCH_JOB_EXECUTION_PARAMS`
 - `BATCH_STEP_EXECUTION`
 - `BATCH_JOB_EXECUTION_CONTEXT`
 - `BATCH_STEP_EXECUTION_CONTEXT`
-  **Trazabilidad completa:**
 - Cada transacción tiene UUID único
 - Timestamp en todas las operaciones
 - Balance anterior y posterior registrado
 - Relación con cliente y cuenta

Resultados Obtenidos - MÉTRICAS REALES

Métricas del Proyecto

Resultado	Planificado	Resultado Real	Estado
Conceptos del Curso Aplicados	100%	100%	✓
Proyecto Portfolio Completo	1	1 proyecto funcional	✓
APIs REST Documentadas	15+	50+ endpoints con Swagger	✓ SUPERADO
Coverage de Testing Alcanzado	85%	87%	✓ SUPERADO
Líneas de Código Tests	No especificado	3775 líneas	✓
Archivos de Test	No especificado	15 archivos	✓
Patrones de Diseño	Polimorfismo	Strategy, Factory, Event-Driven, Repository, DTO	✓ SUPERADO

Conceptos del Curso Cubiertos - VERIFICACIÓN REAL

Inyección de Dependencias ✓

Planificado:

- Constructor injection en todos los services
- Uso de interfaces para desacoplamiento
- Configuración con anotaciones Spring

Polimorfismo ✓

Planificado:

- Calculadoras de interés por tipo de cuenta
 - Strategy pattern implementado
 - Interfaces con múltiples implementaciones
-

Spring Batch

Implementado:

-  **Jobs con múltiples steps:**
 - Job: monthlyInterestJob
 - Step 1: calculateAndApplyInterestStep (Reader → Processor → Writer)
 - Step 2: publishEventsStep (Reader → Processor → Writer)
 - JobExecutionListener: BatchJobExecutionMongoListener
 -  **Procesamiento por lotes:**
 - Chunk-oriented processing con chunk size = 10
 - RepositoryItemReader con paginación automática
 - ItemProcessor con lógica de negocio (Factory Pattern)
 - ItemWriter con actualización transaccional
 - Contexto compartido entre steps (ExecutionContext)
 -  **Scheduling y configuración:**
 - spring.batch.jdbc.initialize-schema=always
 - spring.batch.job.enabled=true
 - @ConditionalOnProperty para habilitar/deshabilitar
 - Trigger manual vía REST API (POST /api/batch/monthly-interest)
 - JobLauncher configurado por Spring Boot
 - JobRepository configurado automáticamente
 - PlatformTransactionManager inyectado
-

Conclusión

5 días • 1 proyecto integral • 100% completado 

Este proyecto final ha **cubierto todos los objetivos planificados:**

Preparación Profesional Alcanzada

Los participantes han aplicado exitosamente:

- Todos los conceptos del curso en un contexto bancario realista
- Arquitectura híbrida MySQL + MongoDB en producción

- Event-Driven Architecture con Modulos definidos por Spring
- Batch processing con Spring Batch
- Testing integral con 87% de cobertura
- Patrones de diseño empresariales
- Documentación técnica completa
- Scripts de automatización

Proyecto Demostrable

El repositorio incluye:

- Código fuente completo y funcional
 - Suite de tests automatizados
 - Documentación completa
 - Scripts de deployment con Docker
 - Reportes de coverage con JaCoCo
 - APIs documentadas con Swagger
-

Preparado para: Participantes Academia Fullstack

Fecha de Planificación: Septiembre 2025

Fecha de Completación: 3 Octubre 2025

Duración Real: 5 días

Estado Final:  PROYECTO 100% COMPLETADO Y FUNCIONAL